



Pashov Audit Group

Venus Periphery Security Review

January 8th 2026 - January 10th 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. About Venus Periphery	4
5. Executive Summary	4
6. Findings	5
Low findings	6
[L-01] SwapAndSupply and SwapAndRepay are nonpayable despite native support	6
[L-02] Same-token swap restriction limits legitimate arbitrage opportunity	6
[L-03] Native supply may distribute unintended <code>vToken</code> balances	7
[L-04] SwapAndRepayFull() allows unintended native transfers with no user recovery	7
[L-05] Full-repay swaps accept zero slippage enabling MEV to extract value	8



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. About Venus Periphery

Venus Periphery is an extension of the Venus Protocol ecosystem that contains auxiliary smart contracts used for advanced interactions with the core protocol. SwapRouter allows users to swap arbitrary tokens (including native wrapped BNB) via a SwapHelper and then automatically supply them to, or repay their debts in, supported Venus vToken markets.

5. Executive Summary

A time-boxed security review of the **VenusProtocol/venus-periphery** repository was done by Pashov Audit Group, during which **Said, ExtraCaterpillar, h2134, Hals** engaged to review **Venus Periphery**. A total of 5 issues were uncovered.

Protocol Summary

Project Name	Venus Periphery
Protocol Type	Periphery contracts
Timeline	January 8th 2026 - January 10th 2026

Review commit hash:

- [4487e12ec6c8dbe0d3b39f066e99405373721a84](#)
(VenusProtocol/venus-periphery)

Fixes review commit hash:

- [f042ecd8c22492745c053648fc2020eef8b7a016](#)
(VenusProtocol/venus-periphery)

Scope

[Interfaces.sol](#)[ISwapRouter.sol](#)[SwapRouter.sol](#)



6. Findings

Findings count

Severity	Amount
Low	5
Total findings	5

Summary of findings

ID	Title	Severity	Status
[L-01]	SwapAndSupply and SwapAndRepay are nonpayable despite native support	Low	Resolved
[L-02]	Same-token swap restriction limits legitimate arbitrage opportunity	Low	Acknowledged
[L-03]	Native supply may distribute unintended <code>vToken</code> balances	Low	Resolved
[L-04]	SwapAndRepayFull() allows unintended native transfers with no user recovery	Low	Resolved
[L-05]	Full-repay swaps accept zero slippage enabling MEV to extract value	Low	Acknowledged



Low findings

[L-01] SwapAndSupply and SwapAndRepay are nonpayable despite native support

Description

`swapAndSupply` and `swapAndRepay` are not payable, but `_handleTokenInput` supports `tokenIn == NATIVE_TOKEN_ADDR`. Because the function cannot receive BNB, any attempt to supply native via this path will revert (`msg.value` is always 0). This is inconsistent with `swapAndRepayFull`, which is payable, and can mislead integrators about native support on these entry points.

Consider either marking `swapAndSupply` and `swapAndRepay` as payable, or removing native-token handling from `_handleTokenInput` and also removing payable from `swapAndRepayFull`.

[L-02] Same-token swap restriction limits legitimate arbitrage opportunity

Description

In the `_performSwap()` function, when `tokenIn` equals `tokenOut`, the contract bypasses the swap execution entirely and returns `amountIndirectly`, regardless of whether `swapCallData` is provided.

```
// If tokens are the same, no swap needed
if (tokenIn == tokenOut) {
    if (minAmountOut > 0 && amountIn < minAmountOut) {
        revert InsufficientAmountOut(amountIn, minAmountOut);
    }
    return amountIn;
}
```

This design assumes that swapping identical tokens is unnecessary. However, it unintentionally blocks legitimate arbitrage strategies where users may intend to execute complex DeFi operations—such as cross-DEX arbitrage, fee optimization, or flash loan maneuvers—even when input and output tokens are the same.

Recommendation

Allow the swap to proceed if `swapCallData` is non-empty, indicating explicit user intent.



[L-03] Native supply may distribute unintended `vToken` balances

Description

In `SwapRouter._supply()`, when supplying to the native `vToken` (`vBNB`), the router mints `vBNB` to itself and then transfers its entire `vBNB` balance to the caller.

If the router already holds `vBNB` (for example, due to accidental transfers or previous dust), the next caller will receive more `vBNB` than what was minted in their transaction.

```
function _supply(
    address vToken,
    address underlyingToken,
    uint256 amount
) internal returns (uint256 amountSupplied) {
    if (vToken == address(NATIVE_VTOKEN)) {
        // Handle native token supply
        IWBNB(underlyingToken).withdraw(amount);
        NATIVE_VTOKEN.mint{ value: amount }();
        amountSupplied = amount;

        // Transfer vBNB tokens to user (only needed for NATIVE_VTOKEN since it doesn't
        support mintBehalf)
    @531>         uint256 vTokenBalance = IERC20Upgradeable(vToken).balanceOf(address(this));
    @532>         if (vTokenBalance > 0) {
    @533>             IERC20Upgradeable(vToken).safeTransfer(msg.sender, vTokenBalance);
        }
    } else {
        //...
    }
}
```

Recommendation:

Track and transfer only the delta between the router's `vBNB` balance before and after minting, instead of forwarding the entire balance.

[L-04] `SwapAndRepayFull()` allows unintended native transfers with no user recovery

Description

In `SwapRouter.swapAndRepayFull()`, the function is marked as payable even when the repayment flow uses an ERC-20 (`tokenIn`). When `tokenIn` is not the native address, the function ignores `msg.value` entirely. Any native BNB sent alongside such a call is neither wrapped nor refunded, and remains stuck in the router until recovered by the owner via the `sweepNative()` function.

Recommendation: Update `swapAndRepayFull()` to revert if `msg.value > 0` when `tokenIn` is not the native token.



[L-05] Full-repay swaps accept zero slippage enabling MEV to extract value

Description

`swapAndRepayFull` and `swapNativeAndRepayFull` call `_performSwap` with `minAmountOut = 0`, so the swap has no price protection. An attacker can front-run by repaying part of the victim's debt, lowering the `debtAmount` or required output.

```
function swapAndRepayFull(
    address vToken,
    address tokenIn,
    uint256 maxAmountIn,
    bytes calldata swapCallData
) external payable nonReentrant {
    if (maxAmountIn == 0) revert ZeroAmount();
    _validateVToken(vToken);

    // Get user's current debt
    >>> uint256 debtAmount = IVToken(vToken).borrowBalanceCurrent(msg.sender);
    if (debtAmount == 0) revert ZeroAmount();

    address tokenOut = _getUnderlyingToken(vToken);

    // Handle input token transfer
    uint256 actualAmountIn = _handleTokenInput(tokenIn, maxAmountIn);

    // Perform swap - no minAmountOut since we need exact debt amount
    uint256 amountOut = _performSwap(tokenIn, tokenOut, actualAmountIn, 0, swapCallData);
    // Ensure we have enough to cover debt
    >>> if (amountOut < debtAmount) revert InsufficientAmountOut(amountOut, debtAmount);

    // Repay full debt
    uint256 amountRepaid = _repay(vToken, tokenOut, amountOut);

    emit SwapAndRepay(msg.sender, vToken, tokenIn, tokenOut, actualAmountIn, amountOut,
        amountRepaid);
}
```

Then immediately sandwich the victim's swap (move the pool price up), so the victim overpays for the remaining debt. Finally, back-run to unwind and capture the price impact.

Because the router only requires `amountOut >= debtAmount` and sets `minAmountOut = 0`, the trade executes at the manipulated, worse price. Excess output over the live debt is refunded, but the overpaid input in the swap is lost.

Recommendations

Require caller-provided slippage protection for the full-repay flows, add `minAmountOut`, and pass it to `_performSwap`.